

Concurrency and Distributed Systems

Week 7 – Part 4: Testing Traps, Stress Thinking, and NIO Counterpoint

Dr. Mohamad Aoude

Faculty of Engineering

May 13, 2026

Part 4 Roadmap

- 1 Testing Trap
- 2 Stress-Style Thinking
- 3 NIO Counterpoint
- 4 Demo and Week 8 Bridge

The False Comfort Problem

A test that passes once proves little

Concurrent bugs often depend on timing, scheduling, CPU count, warm-up, and tiny instruction windows.

Week 7 warning

"It passed 1000 times" is not the same as "the code is thread-safe."

Reproduction Strategy

A good reproduction strategy usually combines:

- many iterations,
- a small shared critical window,
- `Thread.yield()` or deliberate scheduling disturbance,
- randomized ordering,
- clear expected and observed outcomes.

JCStress-Style Outcome Classification

Outcome type	Meaning
Acceptable	The result is allowed by the intended contract.
Interesting	The result exposes a race or weak behavior worth studying.
Forbidden	The result must never happen.

What Students Implement

Exercise 3

Classify counter results:

- expected result is acceptable,
- lost updates are interesting,
- impossible values are forbidden.

Async Does Not Mean More Threads

- Blocking thread-per-request uses one blocked thread per waiting connection.
- NIO can multiplex many connections with fewer blocked threads.
- This improves scalability for waiting I/O, not CPU speed.
- The correct comparison is throughput, latency, thread count, and complexity.

Design Comparison

Design	Strength	Risk
Thread per request	simple mental model	too many blocked threads
Bounded pool	protects server	queued latency under load
CompletableFuture	composable completion	missing timeout/fallback
NIO selector	many connections, few threads	harder state management

Run

```
.\run-demo.ps1 03  
.\run-demo.ps1 04  
.\run-demo.ps1 05  
.\run-demo.ps1 07
```

Where Week 7 leads

Week 7 teaches async composition. Week 8 will move toward non-blocking server design and measuring real server behavior.

- Do not confuse async APIs with correctness.
- Do not confuse fewer threads with lower latency.
- Always measure and always expose fallback.